

EC2x&EG9x&EG25-G 系列

QuecOpen GNSS API 参考手册

LTE Standard 模块系列

版本：1.0

日期：2021-02-08

状态：受控文件

上海移远通信技术股份有限公司始终以为客户提供最及时、最全面的服务为宗旨。如需任何帮助，请随时联系我司上海总部，联系方式如下：

上海移远通信技术股份有限公司
上海市闵行区田林路 1016 号科技绿洲 3 期（B 区）5 号楼 邮编：200233
电话：+86 21 51086236 邮箱：info@quectel.com

或联系我司当地办事处，详情请登录：<http://www.quectel.com/cn/support/sales.htm>。

如需技术支持或反馈我司技术文档中的问题，可随时登陆如下网址：
<http://www.quectel.com/cn/support/technical.htm> 或发送邮件至：support@quectel.com。

前言

上海移远通信技术股份有限公司提供该文档内容用以支持其客户的产品设计。客户须按照文档中提供的规范、参数来设计其产品。因未能遵守有关操作或设计规范而造成的损害，上海移远通信技术股份有限公司不承担任何责任。在未声明前，上海移远通信技术股份有限公司有权对该文档进行更新。

免责声明

上海移远通信技术股份有限公司尽力确保开发中功能的完整性、准确性、及时性或效用，但不排除上述功能错误或遗漏的可能。除非其他有效协议另有规定，否则上海移远通信技术股份有限公司对开发中功能的使用不做任何暗示或明示的保证。在适用法律允许的最大范围内，上海移远通信技术股份有限公司不对任何因使用开发中功能而遭受的损失或损害承担责任，无论此类损失或损害是否可以预见。

保密义务

除非上海移远通信技术股份有限公司特别授权，否则我司所提供文档和信息的接收方须对接收的文档和信息保密，不得将其用于除本项目的实施与开展以外的任何其他目的。未经上海移远通信技术股份有限公司书面同意，不得获取、使用或向第三方泄露我司所提供的文档和信息。对于任何违反保密义务、未经授权使用或以其他非法形式恶意使用所述文档和信息的违法侵权行为，上海移远通信技术股份有限公司有权追究法律责任。

版权申明

本文档版权属于上海移远通信技术股份有限公司，任何人未经我司允许而复制转载该文档将承担法律责任。

版权所有 ©上海移远通信技术股份有限公司 2021，保留一切权利。

Copyright © Quectel Wireless Solutions Co., Ltd. 2021.

文档历史

修订记录

版本	日期	作者	变更表述
-	2021-01-07	Navy QIU/ Arno WANG	文档创建
1.0	2021-02-08	Navy QIU/ Arno WANG	受控版本

目录

文档历史	2
目录	3
表格索引	5
1 引言	6
1.1. 适用模块	6
2 GNSS 功能特性介绍	7
2.1. 功能特性	7
2.2. 性能指标	7
3 GNSS 接口函数介绍	9
3.1. 头文件	9
3.2. 参考示例	9
3.3. 枚举	9
3.3.1. E_QL_LOC_NFY_MSG_ID_T	9
3.3.2. E_QL_LOC_DELETE_AIDING_DATA_TYPE_T	10
3.3.3. E_QL_LOC_AGPS_TYPE_T	12
3.3.4. E_QL_LOC_NI_USER_RESPONSE_TYPE_T	13
3.4. 结构体	13
3.4.1. QL_LOC_INJECT_TIME_INTO_T	13
3.4.2. QL_LOC_INJECT_LOCATION_INTO_T	14
3.4.3. QL_LOC_AGPS_DATA_CONN_OPEN_INTO_T	14
3.4.4. QL_LOC_AGPS_SERVER_INTO_T	15
3.4.5. QL_LOC_POS_MODE_INFO_T	15
3.4.6. QL_LOC_LOCATION_INFO_T	16
3.4.7. QL_LOC_NI_RESPONSE_INTO_T	18
3.5. API 详解	18
3.5.1. QL_LOC_Client_Init	19
3.5.2. QL_LOC_Client_Deinit	19
3.5.3. QL_LOC_AddRxIndMsgHandler	20
3.5.3.1. QL_LOC_RxIndMsgHandlerFunc_t	20
3.5.4. QL_LOC_Set_Indications	21
3.5.5. QL_LOC_Start_Navigation	22
3.5.6. QL_LOC_Stop_Navigation	22
3.5.7. QL_LOC_Set_Position_Mode	23
3.5.8. QL_LOC_Get_Current_Location	23
3.5.9. QL_LOC_Delete_Aiding_Data	24
3.5.10. QL_LOC_InjectTime	24
3.5.11. QL_LOC_InjectLocation	25
3.5.12. QL_LOC_Xtra_InjectData	26
3.5.13. QL_LOC_Xtra_InjectFile	26
3.5.14. QL_LOC_Agps_DataConnOpen	27

3.5.15.	QL_LOC_Agps_DataConnClose	27
3.5.16.	QL_LOC_Agps_NfyDataConnFailed	28
3.5.17.	QL_LOC_Agps_SetServer.....	28
3.5.18.	QL_LOC_NI_SetResponse.....	29
3.5.19.	QL_LOC_Agps_UpdateNWAvailability	29
4	示例	31
4.1.	GNSS API 使用步骤	31
4.2.	示例说明	31
4.3.	示例代码.....	33
4.4.	示例编译说明.....	37
5	附录 A 参考文档及术语缩写	38

表格索引

表 1: 适用模块 6

表 2: 性能指标 7

表 3: 接口概览 18

表 4: 参考文档 38

表 5: 术语缩写 38

1 引言

移远通信 LTE Standard EC2x 系列、EG9x 系列和 EG25-G 模块支持 QuecOpen®方案；QuecOpen®是开源的基于 Linux 的嵌入式开发平台，可简化 IoT 应用的软件设计和开发过程。有关 QuecOpen®的详细信息，请参考[文档\[1\]](#)。

本文档主要介绍如何通过使用移远通信 EC2x 系列、EG9x 系列和 EG25-G QuecOpen 模块 SDK 中提供的 GNSS API 接口函数实现模块的 GNSS 相关功能。

GNSS 即全球导航卫星系统，泛指所有的卫星导航系统，包括全球卫星导航、区域导航以及增强型卫星导航。全球卫星导航系统有 GPS、GLONASS、BeiDou、Galileo 等；区域导航系统有日本的 QZSS 及印度的 IRNSS；增强型卫星导航系统有 WAAS、SDCM、EGNOS、MSAS、GAGAN。

1.1. 适用模块

表 1：适用模块

模块系列	模块
EC2x 系列	EC25 系列
	EC21 系列
	EC20 R2.1
	EC20-CN
EG9x 系列	EG95 系列
	EG91 系列
EG25-G	EG25-G

2 GNSS 功能特性介绍

移远通信 EC2x 系列、EG9x 系列和 EG25-G QuecOpen 模块采用 IZat Gen8C GNSS 引擎，支持 GPS、BeiDou、GLONASS、Galileo 等导航系统，支持 SUPL、XTRA 快速定位技术等，使定位更快速、更准确。

本章节主要介绍移远通信 EC2x 系列、EG9x 系列和 EG25-G QuecOpen 模块的 GNSS 功能特性和性能指标。

2.1. 功能特性

- 支持星基增强系统（SBAS），包括 WAAS、EGNOS、MSAS、GAGAN 等。
- 支持 XTRA 快速定位技术。
- 支持 AGPS 辅助定位技术，例如 SUPL。
- 支持多星座定位功能，包括 GPS、GLONASS、BeiDou、Galileo、QZSS 等。
- 支持单点定位、差分定位（DGPS）。
- 支持多种频率输出定位信息，1 Hz、2 Hz、5 Hz、10 Hz 等。

2.2. 性能指标

表 2：性能指标

指标	性能	备注
2D 定位精度（50%、68%、95%）	< 2 米、< 2.5 米、< 5 米	开阔地带； 定位模式：Stand-alone 模式
3D 定位精度（50%、68%、95%）	< 2.5 米、< 3 米、< 6 米	开阔地带； 定位模式：Stand-alone 模式
同时跟踪通道数	40	-
冷启动 TTFF	29 秒	开阔地带； 定位模式：Stand-alone 模式
温启动 TTFF	27 秒	开阔地带； 定位模式：Stand-alone 模式

热启动 TTFF	1 秒	开阔地带； 定位模式：Stand-alone 模式
失锁 30 秒后重捕获时间	≈ 1 秒	开阔地带
失锁 5 分钟后重捕获时间	≈ 2 秒	开阔地带
捕获灵敏度（冷启动，95%）	> -149 dBm	冷启动，300 秒超时
跟踪灵敏度	> -163 dBm	定位模式：Stand-alone 或 MSB 模式
速度精度（68%、95%）	0.15 米/秒、0.3 米/秒	直线行驶，30 米/秒
航向精度（68%、95%）	0.2 度、0.5 度	直线行驶，30 米/秒
最大速度	1852 千米/时	

备注

有关定位模式详情，请参考第 3.4.5 章。

3 GNSS 接口函数介绍

3.1. 头文件

接口头文件位于 `ql-ol-sdk/ql-ol-extsdk/include/ql_mcm_gps.h`。

3.2. 参考示例

本文档接口示例位于 `ql-ol-sdk/ql-ol-extsdk/example/API/api_test_main.c`。

3.3. 枚举

3.3.1. E_QL_LOC_NFY_MSG_ID_T

数据 ID 及其对应数据结构的枚举定义如下：

```
typedef enum
{
    E_QL_LOC_NFY_MSG_ID_STATUS_INFO = 0,           /**< pv_data = &E_QL_LOC_ST
    ATUS_VALUE_T */
    E_QL_LOC_NFY_MSG_ID_LOCATION_INFO,             /**< pv_data = &QL_LOC_LOC
    ATION_INFO_T */
    E_QL_LOC_NFY_MSG_ID_SV_INFO,                   /**< pv_data = &QL_LOC_SV_ST
    ATUS_T */
    E_QL_LOC_NFY_MSG_ID_NMEA_INFO,                 /**< pv_data = &QL_LOC_NME
    A_INFO_T */
    E_QL_LOC_NFY_MSG_ID_CAPABILITIES_INFO,         /**< pv_data = &E_QL_LOC_CA
    PABILITIES_T */
    E_QL_LOC_NFY_MSG_ID_AGPS_STATUS,               /**< pv_data = &QL_LOC_AGP
    S_STATUS_T */
    E_QL_LOC_NFY_MSG_ID_NI_NOTIFICATION,          /**< pv_data = &QL_LOC_NI_NO
    TIFICATION_INTOTO_T */
    E_QL_LOC_NFY_MSG_ID_XTRA_REPORT_SERVER,        /**< pv_data = &QL_LOC_X
```

```
TRA_REPORT_SERVER_INT0_T */
}E_QL_LOC_NFY_MSG_ID_T;
```

● 参数

参数	描述
<i>E_QL_LOC_NFY_MSG_ID_STATUS_INFO</i>	GNSS 状态数据
<i>E_QL_LOC_NFY_MSG_ID_LOCATION_INFO</i>	定位数据
<i>E_QL_LOC_NFY_MSG_ID_SV_INFO</i>	可见卫星数据
<i>E_QL_LOC_NFY_MSG_ID_NMEA_INFO</i>	NMEA 语句数据
<i>E_QL_LOC_NFY_MSG_ID_CAPABILITIES_INFO</i>	定位方式
<i>E_QL_LOC_NFY_MSG_ID_AGPS_STATUS</i>	AGPS 状态数据
<i>E_QL_LOC_NFY_MSG_ID_NI_NOTIFICATION</i>	NI 通知
<i>E_QL_LOC_NFY_MSG_ID_XTRA_REPORT_SERVER</i>	XTRA 服务器数据

3.3.2. E_QL_LOC_DELETE_AIDING_DATA_TYPE_T

待删除的辅助定位数据类型枚举定义如下：

```
typedef enum
{
    E_QL_LOC_DELETE_EPHEMERIS      = (1 << 0),    /**< Delete ephemeris data. */
    E_QL_LOC_DELETE_ALMANAC        = (1 << 1),    /**< Delete almanac data. */
    E_QL_LOC_DELETE_POSITION       = (1 << 2),    /**< Delete position data. */
    E_QL_LOC_DELETE_TIME           = (1 << 3),    /**< Delete time data. */
    E_QL_LOC_DELETE_IONO           = (1 << 4),    /**< Delete IONO data. */
    E_QL_LOC_DELETE_UTC            = (1 << 5),    /**< Delete UTC data. */
    E_QL_LOC_DELETE_HEALTH         = (1 << 6),    /**< Delete health data. */
    E_QL_LOC_DELETE_SVDIR          = (1 << 7),    /**< Delete SVDIR data. */
    E_QL_LOC_DELETE_SVSTEER        = (1 << 8),    /**< Delete SVSTEER data. */
    E_QL_LOC_DELETE_SADATA         = (1 << 9),    /**< Delete SA data. */
    E_QL_LOC_DELETE_RTI            = (1 << 10),   /**< Delete RTI data. */
    E_QL_LOC_DELETE_CELLDB_INFO    = (1 << 11),   /**< Delete cell DB information.
*/
    E_QL_LOC_DELETE_ALMANAC_CORR    = (1 << 12),   /**< Delete almanac correction
data. */
    E_QL_LOC_DELETE_FREQ_BIAS_EST  = (1 << 13),   /**< Delete frequency bias estim
ate. */
}
```

```

E_QL_LOC_DELETE_EPHEMERIS_GLO = (1 << 14), /**< Delete ephemeris GLO dat
a. */
E_QL_LOC_DELETE_ALMANAC_GLO = (1 << 15), /**< Delete almanac GLO data.
*/
E_QL_LOC_DELETE_SVDIR_GLO = (1 << 16), /**< Delete SVDIR GLO data. */
E_QL_LOC_DELETE_SVSTEER_GLO = (1 << 17), /**< Delete SVSTEER GLO dat
a. */
E_QL_LOC_DELETE_ALMANAC_CORR_GLO= (1 << 18), /**< Delete almanac correctio
n GLO data. */
E_QL_LOC_DELETE_TIME_GPS = (1 << 19), /**< Delete time GPS data. */
E_QL_LOC_DELETE_TIME_GLO = (1 << 20), /**< Delete time GLO data. */
E_QL_LOC_DELETE_ALL = 0xFFFFFFFF, /**< Delete all location data. */
}E_QL_LOC_DELETE_AIDING_DATA_TYPE_T;

```

● 参数

参数	描述
<i>E_QL_LOC_DELETE_EPHEMERIS</i>	删除星历数据
<i>E_QL_LOC_DELETE_ALMANAC</i>	删除历书数据
<i>E_QL_LOC_DELETE_POSITION</i>	删除定位数据
<i>E_QL_LOC_DELETE_TIME</i>	删除时间数据
<i>E_QL_LOC_DELETE_IONO</i>	删除电离层校正数据
<i>E_QL_LOC_DELETE_UTC</i>	删除 UTC 数据
<i>E_QL_LOC_DELETE_HEALTH</i>	删除健康度数据
<i>E_QL_LOC_DELETE_SVDIR</i>	删除可用卫星方位数据
<i>E_QL_LOC_DELETE_SVSTEER</i>	删除可用卫星转速数据
<i>E_QL_LOC_DELETE_SADATA</i>	删除卫星数据
<i>E_QL_LOC_DELETE_RTI</i>	删除实时集成数据
<i>E_QL_LOC_DELETE_CELLDB_INFO</i>	删除小区数据库数据
<i>E_QL_LOC_DELETE_ALMANAC_CORR</i>	删除历书校正数据
<i>E_QL_LOC_DELETE_FREQ_BIAS_EST</i>	删除频偏估计数据
<i>E_QL_LOC_DELETE_EPHEMERIS_GLO</i>	删除 GLONASS 星历数据
<i>E_QL_LOC_DELETE_ALMANAC_GLO</i>	删除 GLONASS 历书数据

<code>E_QL_LOC_DELETE_SVDIR_GLO</code>	删除 GLONASS 可用卫星方位数据
<code>E_QL_LOC_DELETE_SVSTEER_GLO</code>	删除 GLONASS 可用卫星转速数据
<code>E_QL_LOC_DELETE_ALMANAC_CORR_GLO</code>	删除 GLONASS 历书校正数据
<code>E_QL_LOC_DELETE_TIME_GPS</code>	删除 GPS 时间数据
<code>E_QL_LOC_DELETE_TIME_GLO</code>	删除 GLONASS 时间数据
<code>E_QL_LOC_DELETE_ALL</code>	删除所有数据

3.3.3. E_QL_LOC_AGPS_TYPE_T

AGPS 协议类型枚举定义如下：

```
typedef enum
{
    E_QL_LOC_AGPS_TYPE_INVALID        = -1,    /**< Invalid. */
    E_QL_LOC_AGPS_TYPE_ANY            = 0,      /**< Any. */
    E_QL_LOC_AGPS_TYPE_SUPL          = 1,      /**< SUPL. */
    E_QL_LOC_AGPS_TYPE_C2K           = 2,      /**< C2K. */
    E_QL_LOC_AGPS_TYPE_WWAN_ANY      = 3,      /**< WWAN any. */
    E_QL_LOC_AGPS_TYPE_WIFI          = 4,      /**< Wi-Fi. */
    E_QL_LOC_AGPS_TYPE_SUPL_ES       = 5,      /**< SUPL_ES. */
}E_QL_LOC_AGPS_TYPE_T;
```

● 参数

参数	描述
<code>E_QL_LOC_AGPS_TYPE_INVALID</code>	无效索引
<code>E_QL_LOC_AGPS_TYPE_ANY</code>	任意 AGPS 协议
<code>E_QL_LOC_AGPS_TYPE_SUPL</code>	SUPL 协议
<code>E_QL_LOC_AGPS_TYPE_C2K</code>	C2K
<code>E_QL_LOC_AGPS_TYPE_WWAN_ANY</code>	任意 WWAN
<code>E_QL_LOC_AGPS_TYPE_WIFI</code>	Wi-Fi 协议
<code>E_QL_LOC_AGPS_TYPE_SUPL_ES</code>	SUPL_ES 协议

3.3.4. E_QL_LOC_NI_USER_RESPONSE_TYPE_T

NI 用户响应类型信息枚举定义如下：

```
typedef enum
{
    E_QL_LOC_NI_RESPONSE_ACCEPT      = 1,      /**< Accept. */
    E_QL_LOC_NI_RESPONSE_DENY       = 2,      /**< Deny. */
    E_QL_LOC_NI_RESPONSE_NORESP     = 3,      /**< No response. */
}E_QL_LOC_NI_USER_RESPONSE_TYPE_T;
```

● 参数

类型	描述
E_QL_LOC_NI_RESPONSE_ACCEPT	接受响应
E_QL_LOC_NI_RESPONSE_DENY	拒绝响应
E_QL_LOC_NI_RESPONSE_NORESP	无响应

3.4. 结构体

3.4.1. QL_LOC_INJECT_TIME_INTO_T

时间参数结构体定位如下：

```
typedef struct
{
    int64_t time;          /**< Inject time.*/
    int64_t time_reference; /**< Time reference.*/
    int32_t uncertainty;   /**< Uncertainty.*/
}QL_LOC_INJECT_TIME_INTO_T;
```

● 参数

类型	参数	描述
int64_t	<i>time</i>	注入时间。单位：毫秒。
int64_t	<i>time_reference</i>	时间参考。固定为 0。
int32_t	<i>uncertainty</i>	时间精度。固定为 3500。

3.4.2. QL_LOC_INJECT_LOCATION_INTO_T

注入的位置信息结构体定义如下：

```
typedef struct
{
    double latitude; /**< Latitude.*/
    double longitude; /**< Longitude.*/
    float accuracy; /**< Accuracy.*/
}QL_LOC_INJECT_LOCATION_INTO_T;
```

● 参数

类型	参数	描述
double	<i>latitude</i>	经度。单位：度。
double	<i>longitude</i>	纬度。单位：度。
float	<i>accuracy</i>	精度。单位：米。

3.4.3. QL_LOC_AGPS_DATA_CONN_OPEN_INTO_T

AGPS 数据连接信息结构体定义如下：

```
#define QL_LOC_APN_NAME_LENGTH_MAX 100
typedef struct
{
    E_QL_LOC_AGPS_TYPE_T          e_agps_type;
    /**< AGPS type.*/
    char                          apn[QL_LOC_APN_NAME_LENGTH_MAX + 1];
    /**< APN.*/
    E_QL_LOC_AGPS_APN_BEARER_TYPE_T e_bearer_type;
    /**< Bearer type.*/
}QL_LOC_AGPS_DATA_CONN_OPEN_INTO_T;
```

● 参数

类型	参数	描述
E_QL_LOC_AGPS_TYPE_T	<i>e_agps_type</i>	AGPS 协议类型
char	<i>apn</i>	APN 名称
E_QL_LOC_AGPS_APN_BEARER_TYPE_T	<i>e_bearer_type</i>	承载类型

3.4.4. QL_LOC_AGPS_SERVER_INT0_T

AGPS 服务器信息结构体定义如下：

```
typedef struct
{
    E_QL_LOC_AGPS_TYPE_T    e_agps_type;           /**<   AGPS
    type.*/
    char                    host_name[QL_LOC_SEVER_ADDR_LENGTH_MAX + 1];/**<   Host
    name.*/
    uint32_t                port;                   /**<   Port.*/
}QL_LOC_AGPS_SERVER_INT0_T;
```

● 参数

类型	参数	描述
E_QL_LOC_AGPS_TYPE_T	<i>e_agps_type</i>	AGPS 协议类型
char	<i>host_name</i>	服务器地址
uint32_t	<i>port</i>	端口

3.4.5. QL_LOC_POS_MODE_INFO_T

定位配置项结构体定义如下：

```
typedef struct
{
    E_QL_LOC_POS_MODE_T mode;           //定位模式。只支持 Stand-alone 及 MSB 模式。
    E_QL_LOC_POS_RECURRENCE_T recurrence; //定位循环模式。
    uint32_t                min_interval; //定位间隔。单位：毫秒。
    uint32_t                preferred_accuracy; //水平定位精度。单位：米。
    uint32_t                preferred_time; //定位超时时间。单位：毫秒。
}QL_LOC_POS_MODE_INFO_T;
```


● 参数

类型	参数	描述
E_QL_LOC_POS_MODE_T	<i>mode</i>	定位模式。 E_QL_LOC_POS_MODE_STANDALONE: Stand-alone 模式 E_QL_LOC_POS_MODE_MS_BASED: MSB 模式（可加速定位） E_QL_LOC_POS_MODE_MS_ASSISTED: MSA 模式（暂不支持）
E_QL_LOC_POS_RECURRENCE_T	<i>recurrence</i>	定位循环模式。 E_QL_LOC_POS_RECURRENCE_PERIODIC: 循环定位 E_QL_LOC_POS_RECURRENCE_SINGLE: 单次定位
uint32_t	<i>min_interval</i>	定位间隔。单位：毫秒。 有效取值：100、200、500、1000、> 1000
uint32_t	<i>preferred_accuracy</i>	水平定位精度。单位：米。
uint32_t	<i>preferred_time</i>	定位超时时间。单位：毫秒。

3.4.6. QL_LOC_LOCATION_INFO_T

定位数据结构体定义如下：

```
typedef struct
{
    uint32_t    size;
    E_QL_LOC_LOCATION_VALID_FLAG flags;
    E_QL_LOC_ULP_LOCATION_SOURCE position_source;
    double      latitude;
    double      longitude;
    double      altitude;
    float        speed;
    float        bearing;
    float        accuracy;
    int64_t      timestamp;
    int32_t      is_indoor;
    float        floor_number;
    uint32_t     raw_data_len;
    uint8_t      raw_data[QL_LOC_GPS_RAW_DATA_LEN_MAX];
    char          map_url[QL_LOC_GPS_LOCATION_MAP_URL_SIZE + 1];
    uint8_t      map_index[QL_LOC_GPS_LOCATION_MAP_IDX_SIZE];
}QL_LOC_LOCATION_INFO_T;
```

● 参数

类型	参数	描述
uint32_t	size	结构体大小。
E_QL_LOC_LOCATION_VALID_FLAG	flags	数据有效性指示。 掩码为 1 指示当前结构体内数据的有效性，取值为下列掩码的组合： E_QL_LOC_LOCATION_LAT_LONG_VALID: 经纬度数据有效 E_QL_LOC_LOCATION_ALTITUDE_VALID: 海拔有效 E_QL_LOC_LOCATION_SPEED_VALID: 速度有效 E_QL_LOC_LOCATION_BEARING_VALID: 航向有效 E_QL_LOC_LOCATION_ACCURACY_VALID: 定位精度有效 E_QL_LOC_LOCATION_SOURCE_INFO_VALID: 原始信息有效 E_QL_LOC_LOCATION_IS_INDOOR_VALID: 户内模式有效 E_QL_LOC_LOCATION_FLOOR_NUMBE_VALID: 楼层号有效 E_QL_LOC_LOCATION_MAP_URL_VALID: 映射地址有效 E_QL_LOC_LOCATION_MAP_INDEX_VALID: 映射序号有效
E_QL_LOC_ULP_LOCATION_SOURCE	position_source	定位数据来源。
double	latitude	经度。范围：-90~90；单位：度。
double	longitude	纬度。范围：0~180；单位：度。
double	altitude	高度。单位：米。
float	speed	速度。范围：0~540；单位：米/秒。
float	bearing	航向。范围：0~360；单位：度。
float	accuracy	水平精度。单位：米。
int64_t	timestamp	UTC 时间。单位：毫秒。
int32_t	Is_indoor	是否在室内。（依赖于标记 E_QL_LOC_LOCATION_IS_INDOOR_VALID）
float	floor_number	楼层。（依赖标记 E_QL_LOC_LOCATION_IS_INDOOR_VALID）
int32_t	raw_data_len	原始观测量数据长度。范围：0~256。
uint8_t	raw_data	原始观测量数据。（数据无效）
char	map_url	映射 url 地址。（依赖 E_QL_LOC_LOCATION_MAP_URL_VALID）
uint8_t	map_index	映射序号。（依赖 E_QL_LOC_LOCATION_MAP_INDEX_VALID）

3.4.7. QL_LOC_NI_RESPONSE_INTO_T

NI 响应信息结构体定义如下：

```
typedef struct
{
    int32_t notify_id; /**< Notification ID.*/
    E_QL_LOC_NI_USER_RESPONSE_TYPE_T user_resp; /**< User response.*/
}QL_LOC_NI_RESPONSE_INTO_T;
```

● 参数

类型	参数	描述
int32_t	<i>notify_id</i>	通知 ID。
E_QL_LOC_NI_USER_RESPONSE_TYPE_T	<i>user_resp</i>	用户响应。请参考第3.3.4 章。

3.5. API 详解

表 3：接口概览

函数	说明
<i>QL_LOC_Client_Init</i>	初始化 GNSS 服务句柄
<i>QL_LOC_Client_Deinit</i>	注销 GNSS 服务句柄
<i>QL_LOC_AddRxIndMsgHandler</i>	注册 GNSS 数据处理回调函数
<i>QL_LOC_Set_Indications</i>	设置回调信息类型
<i>QL_LOC_Start_Navigation</i>	启动 GNSS
<i>QL_LOC_Stop_Navigation</i>	关闭 GNSS
<i>QL_LOC_Set_Position_Mode</i>	设置定位配置项
<i>QL_LOC_Get_Current_Location</i>	获取当前定位数据
<i>QL_LOC_Delete_Aiding_Data</i>	删除 GNSS 辅助数据
<i>QL_LOC_InjectTime</i>	向 GNSS 注入 UTC 时间信息

<i>QL_LOC_InjectLocation</i>	向 GNSS 注入位置信息
<i>QL_LOC_Xtra_InjectData</i>	向 GNSS 注入 XTRA 辅助数据
<i>QL_LOC_Xtra_InjectFile</i>	向 GNSS 注入 XTRA 文件
<i>QL_LOC_Agps_DataConnOpen</i>	通知 AGPS 数据连接已经打开
<i>QL_LOC_Agps_DataConnClose</i>	通知 AGPS 数据连接已经关闭
<i>QL_LOC_Agps_NfyDataConnFailed</i>	通知 AGPS 数据连接启动失败
<i>QL_LOC_Agps_SetServer</i>	设置 AGPS 服务器地址及端口
<i>QL_LOC_NI_SetResponse</i>	发送 NI 用户响应信息
<i>QL_LOC_Agps_UpdateNWAvailability</i>	更新网络可用状态

备注

1. 本文档描述接口不适用含 QDR & PPE 功能的模块版本，详细信息请联系移远通信技术支持。
2. 禁止在回调函数中调用其他接口。

3.5.1. QL_LOC_Client_Init

该函数用于初始化 GNSS 客户端，创建 GNSS 会话。

● 函数原型

```
int QL_LOC_Client_Init(loc_client_handle_type *ph_loc);
```

● 参数

ph_loc:

[Out] 句柄。初始化 GNSS 客户端并创建 GNSS 会话后返回的句柄，用于后续 GNSS 接口的调用。

● 返回值

0 会话创建成功。
其他值 会话创建失败。

3.5.2. QL_LOC_Client_Deinit

该函数用于注销 GNSS 客户端并释放 GNSS 会话。

- 函数原型

```
int QL_LOC_Client_Deinit(loc_client_handle_type h_loc);
```

- 参数

h_loc:

[In] 句柄。初始化 GNSS 客户端并成功创建 GNSS 会话时返回的句柄。

- 返回值

0 会话注销成功。

其他值 会话注销失败。

3.5.3. QL_LOC_AddRxIndMsgHandler

该函数用于注册 GNSS 数据处理回调函数。注册的回调函数可接收消息由 *QL_LOC_Set_Indications* 确定。

- 函数原型

```
int QL_LOC_AddRxIndMsgHandler(QL_LOC_RxIndMsgHandlerFunc_t handlerPtr, void* contextPtr);
```

- 参数

handlerPtr:

[In] 用于处理 GNSS 数据的回调函数。

contextPtr:

[In] 回调函数需输入的参数。详情请参考第 3.5.3.1 章。

- 返回值

0 注册回调函数成功。

其他值 注册回调函数失败。

3.5.3.1. QL_LOC_RxIndMsgHandlerFunc_t

该回调函数用于处理 GNSS 数据。根据 *QL_LOC_Set_Indications* 和 *QL_LOC_Set_Position_Mode* 的设置，该回调函数将在对应事件发生时上报相应数据。

- 函数原型

```
typedef void (*QL_LOC_RxIndMsgHandlerFunc_t)
(
    loc_client_handle_type h_loc,
    E_QL_LOC_NFY_MSG_ID_T e_msg_id,
    void *pv_data,
    void *contextPtr
);
```

- 参数

h_loc:

[Out] 句柄。初始化 GNSS 客户端并成功创建 GNSS 会话时返回的句柄。

e_msg_id:

[Out] 数据 ID。详情请参考第 3.3.1 章。

pv_data:

[Out] 回调数据内容。详情请参考第 3.3.1 章。

contextPtr:

[Out] 自定义数据。回调标签。

- 返回值

无

3.5.4. QL_LOC_Set_Indications

该函数用于设置回调数据。

- 函数原型

```
int QL_LOC_Set_Indications(loc_client_handle_type h_loc, int bit_mask);
```

- 参数

h_loc:

[In] 句柄。初始化 GNSS 客户端并成功创建 GNSS 会话时返回的句柄。

bit_mask:

[In] 回调数据开关，其定义如下：

LOC_IND_LOCATION_INFO_ON	(1 << 0)	//定位数据
LOC_IND_STATUS_INFO_ON	(1 << 1)	//GNSS 引擎状态数据
LOC_IND_SV_INFO_ON	(1 << 2)	//卫星数据
LOC_IND_NMEA_INFO_ON	(1 << 3)	//NMEA 语句
LOC_IND_CAP_INFO_ON	(1 << 4)	//定位方式
LOC_IND_UTC_TIME_REQ_ON	(1 << 5)	//注入 UTC 时间请求
LOC_IND_XTRA_DATA_REQ_ON	(1 << 6)	//注入 XTRA 数据请求
LOC_IND_AGPS_DATA_CONN_CMD_REQ_ON	(1 << 7)	//数据连接请求状态开启
LOC_IND_NI_NFY_USER_RESP_REQ_ON	(1 << 8)	//NI 通知用户回应请求

- 返回值

0 设置回调数据标志位成功。

其他值 设置回调数据标志位失败。

3.5.5. QL_LOC_Start_Navigation

该函数用于启动 GNSS，开始获取导航数据。

- 函数原型

```
int QL_LOC_Start_Navigation(loc_client_handle_type h_loc);
```

- 参数

h_loc:

[In] 句柄。初始化 GNSS 客户端并成功创建 GNSS 会话时返回的句柄。

- 返回值

0 GNSS 启动成功。

其他值 GNSS 启动失败。

3.5.6. QL_LOC_Stop_Navigation

该函数用于关闭 GNSS，停止获取导航数据。

- 函数原型

```
int QL_LOC_Stop_Navigation(loc_client_handle_type h_loc);
```

- 参数

h_loc:

[In] 句柄。初始化 GNSS 客户端并成功创建 GNSS 会话时返回的句柄。

- 返回值

0 GNSS 关闭成功。

其他值 GNSS 关闭失败。

3.5.7. QL_LOC_Set_Position_Mode

该函数用于设置定位配置项，例如导航模式、获取数据间隔、精度等。

- 函数原型

```
int QL_LOC_Set_Position_Mode(loc_client_handle_type h_loc, QL_LOC_POS_MODE_INFO_T *pt_mode);
```

- 参数

h_loc:

[In] 句柄。初始化 GNSS 客户端并成功创建 GNSS 会话时返回的句柄。

pt_mode:

[In] 定位配置项。详情请参考第3.4.5章。

- 返回值

0 设置成功。

其他值 设置失败。

3.5.8. QL_LOC_Get_Current_Location

该函数用于获取当前定位数据，若在指定时间内未获取到定位数据则返回超时。

- 函数原型

```
int QL_LOC_Get_Current_Location(loc_client_handle_type h_loc, QL_LOC_LOCATION_INFO_T *pt_loc_info, int timeout_sec);
```

- 参数

h_loc:

[In] 句柄。初始化 GNSS 客户端并成功创建 GNSS 会话时返回的句柄。

pt_loc_info:

[Out] 定位数据。请参考第3.4.6章。

timeout_sec:

[In] 定位超时时间。单位：毫秒。

- 返回值

0 获取成功。

-2 超时。

3.5.9. QL_LOC_Delete_Aiding_Data

该函数用于删除 GNSS 辅助数据。

- 函数原型

```
int QL_LOC_Delete_Aiding_Data( loc_client_handle_type h_loc, E_QL_LOC_DELETE_AIDING_DATA_TYPE_T flags);
```

- 参数

h_loc:

[In] 句柄。初始化 GNSS 客户端并成功创建 GNSS 会话时返回的句柄。

flags:

[In] 删除指定类型的数据。请参考第3.3.2章。

- 返回值

0 删除成功。

其他值 删除失败。

备注

删除数据需要一定时间，故调用该函数后应延迟 1~3 秒左右时间后，方可执行其他函数。

3.5.10. QL_LOC_InjectTime

该函数用于向 GNSS 引擎中注入 UTC 时间以判断注入的 XTRA 数据是否有效。应在调用 *QL_LOC_Start_Navigation* 启动 GNSS 前调用该函数注入 UTC 时间，如此可加快定位时间。

- 函数原型

```
int QL_LOC_InjectTime( loc_client_handle_type h_loc, QL_LOC_INJECT_TIME_INTO_T *pt_info);
```

- 参数

h_loc:

[In] 句柄。初始化 GNSS 客户端并成功创建 GNSS 会话时返回的句柄。

pt_info:

[In] 注入的 UTC 时间数据。单位：毫秒。请参考第3.4.1 章。

- 返回值

0 注入 UTC 时间成功。

其他值 注入 UTC 时间失败。

备注

需保证注入的时间与当前 UTC 时间误差小于 10 秒，否则会导致定位时间增加。

3.5.11. QL_LOC_InjectLocation

该函数用于向 GNSS 引擎注入位置信息以实现快速定位。

- 函数原型

```
int QL_LOC_InjectLocation( loc_client_handle_type h_loc, QL_LOC_INJECT_LOCATION_INTO_T *pt_info);
```

- 参数

h_loc:

[In] 句柄。初始化 GNSS 客户端并成功创建 GNSS 会话时返回的句柄。

pt_info:

[In] 注入的位置信息。请参考第3.4.2 章。

- 返回值

0 注入位置信息成功。

其他值 注入位置信息失败。

3.5.12. QL_LOC_Xtra_InjectData

该函数用于向 GNSS 引擎注入 XTRA 辅助数据。

- 函数原型

```
Int QL_LOC_Xtra_InjectData(loc_client_handle_type h_loc, char *data, int length);
```

- 参数

h_loc:

[In] 句柄。初始化 GNSS 客户端并成功创建 GNSS 会话时返回的句柄。

data:

[In] XTRA 数据。

length:

[In] XTRA 数据长度。

- 返回值

0 注入 XTRA 数据成功。

其他值 注入 XTRA 数据失败。

备注

模块支持 IPC 机制，由于 IPC 机制限制，当前支持注入的 XTRA 数据最大长度为 0xFC00。

3.5.13. QL_LOC_Xtra_InjectFile

该函数用于向 GNSS 引擎注入 XTRA 文件。

- 函数原型

```
int QL_LOC_Xtra_InjectFile( loc_client_handle_type h_loc, char *filename);
```

- 参数

h_loc:

[In] 句柄。初始化 GNSS 客户端并成功创建 GNSS 会话时返回的句柄。

filename:

[In] XTRA 文件完整路径。

- 返回值

0 注入 XTRA 文件成功。
其他值 注入 XTRA 文件失败。

备注

模块支持 IPC 机制，由于 IPC 机制限制，当前支持注入的 XTRA 文件最大长度为 0xFC00。

3.5.14. QL_LOC_Agps_DataConnOpen

该函数用于通知 AGPS 数据连接已经打开。

- 函数原型

```
int QL_LOC_Agps_DataConnOpen(loc_client_handle_type h_loc, QL_LOC_AGPS_
DATA_CONN_OPEN_INTOT *pt_info);
```

- 参数

h_loc:

[In] 句柄。初始化 GNSS 客户端并成功创建 GNSS 会话时返回的句柄。

pt_info:

[In] 数据连接服务器信息。请参考第 3.4.3 章。

- 返回值

0 通知成功。
其他值 通知失败。

3.5.15. QL_LOC_Agps_DataConnClose

该函数用于通知 AGPS 数据连接已经关闭。

- 函数原型

```
int QL_LOC_Agps_DataConnClose(loc_client_handle_type h_loc, E_QL_LOC_AGPS_TYPE
_T atype);
```

- 参数

h_loc:

[In] 句柄。初始化 GNSS 客户端并成功创建 GNSS 会话时返回的句柄。

atype:

[In] AGPS 协议类型。请参考第 3.3.3 章。

- 返回值

0 通知成功。

其他值 通知失败。

3.5.16. QL_LOC_Agps_NfyDataConnFailed

该函数用于通知 AGPS 数据连接启动失败。

- 函数原型

```
int QL_LOC_Agps_NfyDataConnFailed(loc_client_handle_type h_loc, E_QL_LOC_AGPS_TY
PE_T atype);
```

- 参数

h_loc:

[In] 句柄。初始化 GNSS 客户端并成功创建 GNSS 会话时返回的句柄。

atype:

[In] AGPS 协议类型。请参考第 3.3.3 章。

- 返回值

0 通知成功。

其他值 通知失败。

3.5.17. QL_LOC_Agps_SetServer

该函数用于设置 AGPS 服务器地址及端口。

- 函数原型

```
int QL_LOC_Agps_SetServer(loc_client_handle_type h_loc, QL_LOC_AGPS_SERVER_INT
O_T *pt_info);
```

- 参数

h_loc:

[In] 句柄。初始化 GNSS 客户端并成功创建 GNSS 会话时返回的句柄。

pt_info:

[In] AGPS 服务器地址及端口。请参考第 3.4.4 章。

- 返回值

0 设置成功。
其他值 设置失败。

3.5.18. QL_LOC_NI_SetResponse

该函数用于发送 NI 用户响应信息。

- 函数原型

```
int QL_LOC_NI_SetResponse(loc_client_handle_type h_loc, QL_LOC_NI_RESPONSE_
INTO_T *pt_info);
```

- 参数

h_loc:

[In] 句柄。初始化 GNSS 客户端并成功创建 GNSS 会话时返回的句柄。

pt_info:

[In] AGPS NI 响应信息。请参考第 3.4.7 章。

- 返回值

0 发送成功。
其他值 发送失败。

3.5.19. QL_LOC_Agps_UpdateNWAvailability

该函数用于更新网络可用状态。

- 函数原型

```
int QL_LOC_Agps_UpdateNWAvailability(loc_client_handle_type h_loc, int available, const c
har *apn);
```

- 参数

h_loc:

[In] 句柄。初始化 GNSS 客户端并成功创建 GNSS 会话时返回的句柄。

available:

[In] 网络是否可用。

apn:

[In] 接入点名称。

- 返回值

0 更新成功。

其他值 更新失败。

4 示例

4.1. GNSS API 使用步骤

QuecOpen SDK 提供示例 `example/API/api_test_main.c` 可用于参考。以下介绍上述 API 的使用步骤。

- 使用案例一：通过回调函数将相关信息上报至应用程序
 1. 调用 `QL_LOC_Client_Init` 初始化 GNSS 客户端并创建 GNSS 会话。
 2. 调用 `QL_LOC_AddRxIndMsgHandler(pf_cb)` 注册 GNSS 数据处理回调函数。
 3. 调用 `QL_LOC_Set_Indications` 设置回调数据。
 4. 调用 `QL_LOC_Set_Position_Mode` 设置定位配置项。
 5. 调用 `QL_LOC_Start_Navigation` 启动 GNSS，开始获取导航数据。
 6. 处理回调函数 `QL_LOC_RxIndMsgHandlerFunc_t` 返回的事件。
 7. 调用 `QL_LOC_Stop_Navigation` 关闭 GNSS，停止获取导航数据。
 8. 调用 `QL_LOC_Client_Deinit` 注销 GNSS 客户端并释放 GNSS 会话。
- 使用案例二：主动获取一次位置信息
 1. 调用 `QL_LOC_Client_Init` 初始化 GNSS 客户端并创建 GNSS 会话。
 2. 调用 `QL_LOC_AddRxIndMsgHandler(pf_cb)` 注册 GNSS 数据处理回调函数。（此步骤可选）
 3. 调用 `QL_LOC_Set_Indications` 设置回调数据。设置 `bit_mask = LOC_IND_LOCATION_INFO_ON`。
 4. 调用 `QL_LOC_Set_Position_Mode` 设置定位循环模式为单次定位。
 5. 调用 `QL_LOC_Get_Current_Location` 获取当前定位数据。若超时，模块返回当前位置信息或上次存储的位置信息。
 6. 调用 `QL_LOC_Client_Deinit` 注销 GNSS 客户端并释放 GNSS 会话。

4.2. 示例说明

执行如下命令运行示例程序 `example_gps`。

```
root@mdm9607-perf:/# ./example_gps
```


运行成功后会有如下信息打印:

```

root@mdm9607-perf:/# ./example_gps
===== gps test start =====
please input test mode(0: sync_get_position_once, other:get_gps_info_by_cb): 1
Starting MCM RIL Services: done
[QL_MCM_Client_Init 529]: mcm_client_init ret=0x2 with h_mcm=0x0 ==> Sleep 2s and R
etry !
[QL_MCM_Client_Init 529]: mcm_client_init ret=0x2 with h_mcm=0x0 ==> Sleep 2s and R
etry !
[QL_MCM_Client_Init 536]: Client initialized successfully 0x3
[QL_MCM_Client_Init 546]: mcm_client_init start up required service!
[ql_mcm_async_cb 252]: #####h_mcm=0x3 msg_id=0x800
[ql_mcm_client_srv_updown_async_cb 33]: #####h_mcm=0x3 msg_id=0x800
[loc_ind_cb 22]:
===== mcmlocservice UP ! =====
QL_LOC_Client_Init ret 0 with h_loc=3
QL_LOC_AddRxIndMsgHandler ret 0
Please input indication bitmask(NiNfy|AGPS|XTRA|UTC|CAP|NMEA|SV|Status|Location):
511 //511=0x1FF=01 1111 1111, 表示打开所有 bit
[ql_mcm_ind_cb 133]: #####h_mcm=0x3 msg_id=0x312
[ql_loc_rx_ind_msg_cb 8]: e_msg_id=4
[ql_mcm_ind_cb 133]: #####h_mcm=0x3 msg_id=0x312
[ql_loc_rx_ind_msg_cb 8]: e_msg_id=4
QL_LOC_Set_Indications ret 0
QL_LOC_Set_Position_Mode ret 0
QL_LOC_Start_Navigation ret=0
Wait and handle event ! You can input -1 to exit): [ql_mcm_ind_cb 133]: #####h_mcm=0x3
msg_id=0x30f
[ql_loc_rx_ind_msg_cb 8]: e_msg_id=0
[ql_mcm_ind_cb 133]: #####h_mcm=0x3 msg_id=0x30f
[ql_loc_rx_ind_msg_cb 8]: e_msg_id=0
[ql_mcm_ind_cb 133]: #####h_mcm=0x3 msg_id=0x30f
[ql_loc_rx_ind_msg_cb 8]: e_msg_id=0
[ql_mcm_ind_cb 133]: #####h_mcm=0x3 msg_id=0x310
[ql_loc_rx_ind_msg_cb 8]: e_msg_id=2
[ql_mcm_ind_cb 133]: #####h_mcm=0x3 msg_id=0x311
[ql_loc_rx_ind_msg_cb 8]: e_msg_id=3
NMEA info: timestamp=315964862708, length=17, nmea=$GPGSV,1,1,0,*65
[ql_mcm_ind_cb 133]: #####h_mcm=0x3 msg_id=0x311
[ql_loc_rx_ind_msg_cb 8]: e_msg_id=3
NMEA info: timestamp=315964862709, length=17, nmea=$GLGSV,1,1,0,*79
[ql_mcm_ind_cb 133]: #####h_mcm=0x3 msg_id=0x311
[ql_loc_rx_ind_msg_cb 8]: e_msg_id=3

```

```
NMEA info: timestamp=315964862709, length=29, nmea=$GPGSA,A,1,,,,,,,,,,,,,*1E
[ql_mcm_ind_cb 133]: #####h_mcm=0x3 msg_id=0x311
[ql_loc_rx_ind_msg_cb 8]: e_msg_id=3
NMEA info: timestamp=315964862709, length=24, nmea=$GPVTG,,T,,M,,N,,K,N*2C
[ql_mcm_ind_cb 133]: #####h_mcm=0x3 msg_id=0x311
[ql_loc_rx_ind_msg_cb 8]: e_msg_id=3
NMEA info: timestamp=315964862710, length=24, nmea=$GPRMC,,V,,,,,,,,,N*53
[ql_mcm_ind_cb 133]: #####h_mcm=0x3 msg_id=0x311
[ql_loc_rx_ind_msg_cb 8]: e_msg_id=3
NMEA info: timestamp=315964862710, length=25, nmea=$GPGGA,,,,,0,,,,,,,,*66 //确认这些信息可以正确输出
```

4.3. 示例代码

```
#include <ql_oe.h>

static void ql_loc_rx_ind_msg_cb(ql_client_handle_type h_loc,
                                E_QL_LOC_NFY_MSG_ID_T e_msg_id,
                                void *pv_data,
                                void *contextPtr)
{
    QL_USER_LOG("e_msg_id=%d\n", e_msg_id);
    switch(e_msg_id)
    {
        //根据对应消息获取相应信息，并做对应处理
        case E_QL_LOC_NFY_MSG_ID_STATUS_INFO:
            break;
        case E_QL_LOC_NFY_MSG_ID_LOCATION_INFO:
        {
            QL_LOC_LOCATION_INFO_T *pt_location = (QL_LOC_LOCATION_INFO_T *)pv_data;

            printf("**** flag=0x%X, Latitude = %f, Longitude=%f, accuracy = %f ****\n",
                pt_location->flags,
                pt_location->latitude,
                pt_location->longitude,
                pt_location->accuracy);

            break;
        }
        case E_QL_LOC_NFY_MSG_ID_SV_INFO:
            break;
        case E_QL_LOC_NFY_MSG_ID_NMEA_INFO:
        {

```

```

        QL_LOC_NMEA_INFO_T *pt_nmea = (QL_LOC_NMEA_INFO_T *)pv_data;

        printf("NMEA info: timestamp=%lld, length=%d, nmea=%s\n",
               pt_nmea->timestamp, pt_nmea->length, pt_nmea->nmea);
        break;
    }
    case E_QL_LOC_NFY_MSG_ID_CAPABILITIES_INFO:
        break;
    case E_QL_LOC_NFY_MSG_ID_AGPS_STATUS:
        break;
    case E_QL_LOC_NFY_MSG_ID_NI_NOTIFICATION:
        break;
    case E_QL_LOC_NFY_MSG_ID_XTRA_REPORT_SERVER:
        break;
}
}

void sync_get_position_once(void)
{
    int          ret          = E_QL_OK;
    int          h_loc        = 0;
    int          bitmask      = 0;
    QL_LOC_POS_MODE_INFO_T t_mode      = {0};
    QL_LOC_LOCATION_INFO_T t_loc_info = {0};
    int          timeout_sec = 60;

    ret = QL_LOC_Client_Init(&h_loc);
    printf("QL_LOC_Client_Init ret %d with h_loc=%d\n", ret, h_loc);

    ret = QL_LOC_AddRxIndMsgHandler(ql_loc_rx_ind_msg_cb, (void*)h_loc);
    printf("QL_LOC_AddRxIndMsgHandler ret %d\n", ret);

    bitmask = 1; //force set to 1 to get location only.

    ret = QL_LOC_Set_Indications(h_loc, bitmask);
    printf("QL_LOC_Set_Indications ret %d\n", ret);

    t_mode.mode          = E_QL_LOC_POS_MODE_STANDALONE;
    t_mode.recurrence    = E_QL_LOC_POS_RECURRENCE_SINGLE;
    t_mode.min_interval  = 1000;
    t_mode.preferred_accuracy = 50;
    t_mode.preferred_time = 90; //参数可根据需要自行调整
    ret = QL_LOC_Set_Position_Mode(h_loc, &t_mode);
    printf("QL_LOC_Set_Position_Mode ret %d\n", ret);

```

```

ret = QL_LOC_Get_Current_Location(h_loc, &t_loc_info, timeout_sec);
printf(" QL_LOC_Get_Current_Location ret %d\n", ret);
if(ret < 0)
{
    if(ret == -2)
    {
        // -2: timeout, may need try again
        printf("QL_LOC_Get_Current_Location timeout, try again!\n");
    }
    else
    {
        printf("QL_LOC_Get_Current_Location Fail, ret %d\n", ret);
    }
}
else
{
    printf("***** Latitude = %lf, Longitude=%lf, altitude=%lf, accuracy = %f *****\n",
        t_loc_info.latitude, t_loc_info.longitude, t_loc_info.altitude, t_loc_info.accuracy);
}

ret = QL_LOC_Client_Deinit(h_loc);
printf("QL_LOC_Client_Deinit ret=%d\n", ret);

return ;
}

void get_gps_info_by_cb(void)
{
    int ret = E_QL_OK;
    int h_loc = 0;
    int bitmask = 0;
    QL_LOC_POS_MODE_INFO_T t_mode = {0};
    QL_LOC_LOCATION_INFO_T t_loc_info = {0};

    ret = QL_LOC_Client_Init(&h_loc);
    printf("QL_LOC_Client_Init ret %d with h_loc=%d\n", ret, h_loc);

    ret = QL_LOC_AddRxIndMsgHandler(ql_loc_rx_ind_msg_cb, (void*)h_loc);
    printf("QL_LOC_AddRxIndMsgHandler ret %d\n", ret);

    printf("Please input indication bitmask(NiNfy|AGPS|XTRA|UTC|CAP|NMEA|SV|Status|Location):\n", ret);
    scanf("%d", &bitmask); //根据需要设置 bitmask，打开对应回调消息

```

```

/* Set what we want callbacks for */
ret = QL_LOC_Set_Indications(h_loc, bitmask);
printf("QL_LOC_Set_Indications ret %d\n", ret);

t_mode.mode                = E_QL_LOC_POS_MODE_STANDALONE;
t_mode.recurrence          = E_QL_LOC_POS_RECURRENCE_PERIODIC;
t_mode.min_interval        = 1000;
t_mode.preferred_accuracy  = 50;
t_mode.preferred_time      = 90; //参数可根据需要自行调整
ret = QL_LOC_Set_Position_Mode(h_loc, &t_mode);
printf("QL_LOC_Set_Position_Mode ret %d\n", ret);

ret = QL_LOC_Start_Navigation(h_loc);
printf("QL_LOC_Start_Navigation ret=%d\n", ret);

while(1)
{
    int finish_flag = 0; //等待消息到来并在回调函数里面处理
    printf("Wait and handle event ! You can input -1 to exit: ");
    scanf("%d", &finish_flag);
    if(finish_flag == -1)
    {
        break;
    }
}
ret = QL_LOC_Stop_Navigation(h_loc);
printf("QL_LOC_Stop_Navigation ret=%d\n", ret);

ret = QL_LOC_Client_Deinit(h_loc);
printf("QL_LOC_Client_Deinit ret=%d\n", ret);
}

int main(int argc, char *argv[])
{
    int mode;

    printf("===== gps test start =====\r\n");
    printf("please input test mode(0: sync_get_position_once, other:get_gps_info_by_cb): ");
    scanf("%d", &mode);

    if(mode == 0)
    {

```

```

        sync_get_position_once();
    }
    else
    {
        get_gps_info_by_cb();
    }
    printf("===== gps test end =====\r\n");
}

```

4.4. 示例编译说明

本章节介绍如何编译单个 `example_voice.c`。

1. 执行如下命令解压 QuecOpen SDK:

```
tar -jxvf ql-ol-sdk.tar.bz2
```

2. 执行如下命令进入 `ql-ol-sdk` 目录:

```
cd ql-ol-sdk
```

3. 执行如下命令配置环境参数:

```
source ql-ol-crostoool/ql-ol-crostoool-env-init
```

备注

需确保 SDK 版本与模块版本一致，否则可能出现错误。

4. 执行如下命令进入示例程序 `example_gps` 的目录:

```
cd ql-ol-extsdk/example/example_gps
```

5. 执行如下命令进行编译:

```
make clean;
make
```

6. 编译成功后生成文件位于 `example_gps` 的目录下。

5 附录 A 参考文档及术语缩写

表 4: 参考文档

序号	文档名称	描述
[1]	Quectel_EC2x&EG9x&EG25-G 系列_QuecOpen_快速开发指导	适用于 EC2x 系列、EG9x 系列和 EG25-G QuecOpen 模块的快速开发指导

表 5: 术语缩写

缩写	英文全称	中文全称
AGPS	Assisted Global Positioning System	辅助全球卫星定位系统
API	Application Programming Interface	应用程序接口
APN	Access Point Name	接入点名称
BeiDou	BeiDou Navigation Satellite System	中国北斗卫星导航系统
DB	Data Base	数据库
DGPS	Differential Global Position System	差分全球定位系统
EGNOS	European Geostationary Navigation Overlay Service	欧洲地球静止导航重叠服务
GAGAN	GPS Aided Geo Augmented Navigation	GPS 辅助 GEO 增强导航
GLONASS	Global Navigation Satellite System in Russia	俄罗斯卫星导航系统格洛纳斯
GNSS	Global Navigation Satellite System	全球导航卫星系统
GPS	Global Positioning System	全球定位系统
IRNSS	Indian Regional Navigation Satellite System	印度区域导航卫星系统
IPC	Inter-Process Communication	进程间通信
MSA	Mobile Station Assisted	移动终端辅助

MSAS	Multi-Functional Satellite Augmentation System	日本多功能卫星增强系统
MSB	Mobile Station Based	基于移动终端
NI	Network Initialed	网络发起请求模式
NMEA	NMEA (National Marine Electronics Association) 0183 Interface Standard	NMEA（美国国家海洋电子协会）0183 接口标准
PPE	Precise Positioning Engine	精确定位引擎
QDR	Qualcomm Dead Reckoning	高通航位推测技术
QZSS	Quasi-Zenith Satellite System	准天顶卫星系统
RTI	Real Time Integration	实时集成数据
SA	Satellite	卫星
SBAS	Satellite-Based Augmentation System	星基增强系统
SDCM	System of Differential Correction and Monitoring	俄罗斯差分校正和监测系统
SDK	Software Development Kit	软件开发工具包
SUPL	Secure User Plane Location	安全用户平面定位协议
SVDIR	Satellites Available Direction	可用卫星方向
SVSTEER	Satellites Available Steer	可用卫星转速
TTFF	Time To First Fix	首次定位时间
UTC	Coordinated Universal Time	协调世界时
WAAS	Wide Area Augmentation System	广域增强系统
WWAN	Wireless Wide Area Network	无线广域网
XTRA	eXTended Receiver Assistance	高通扩展定位辅助数据